Formalizing Analytic Number Theory in Lean 4: Foundations, Practices, and Lessons from the Cathedral Project

Jason Robert Gochanour

April 28, 2026

Abstract

We describe the practical techniques and logical foundations for formalizing analytic number theory in Lean 4 with Mathlib, distilled from the Cathedral project: a 50,623-line formalization of the Nyman–Beurling criterion for the Riemann Hypothesis.

The first half covers foundations: the type-theoretic setting (CIC with extensions), the trust chain from silicon to theorem, the precise logical strength of the formalization (provable in ZFC), and a constructivity analysis showing the converse direction is partially constructive while the forward direction is inherently classical.

The second half covers practice: Mathlib API patterns that worked well (interval integrals, spectral theory), patterns requiring workarounds (infinite sums, measure theory), 5 API gaps requiring axioms, 12 essential lemmas, project structure advice, and common tactic patterns. We document 3 deprecated-API traps and provide a verification checklist for large-scale proof engineering.

Contents

1 Introduction

Every machine-verified proof rests on a stack of trust. This paper examines that stack for the Cathedral, a Lean 4 formalization establishing RH $\iff$ $d_N^2 \to 0$, from two perspectives: the *foundational* (what are we trusting?) and the *practical* (how do we build it?).

Layer	What Must Be Trusted
5	Mathematical axioms (2 on crown path, 104 total)
4	Mathlib library (∼1.4M LOC)
3	Lean kernel axioms (3: `propext`, `choice`, `Quot.sound`)
2	Lean type-checker (∼5,000 LOC of C++ kernel)
1	Compiler, OS, hardware

The Cathedral project encountered the challenge of using complex analysis, real analysis, combinatorics, and linear algebra *together* in a single proof, with consistent type coercions and compatible assumptions. Over 32 days it produced 208 active Lean files (with 100 archived). This paper distills the lessons.

2 The Type-Theoretic Foundation

2.1 CIC with Extensions

Lean 4 is based on the Calculus of Inductive Constructions (CIC) with dependent function types, inductive types, a universe hierarchy, proof irrelevance, and large elimination. Three kernel axioms extend the core:

1. `propext`: $(P \iff Q) \to P = Q$. Essential for set-theoretic reasoning.
2. `Classical.choice`: For any nonempty type A, there exists $a : A$. Implies excluded middle.
3. `Quot.sound`: If $a \sim b$ then $[a] = [b]$ in the quotient type.

The system CIC + `propext` + `choice` + `Quot.sound` is equiconsistent with ZFC.

2.2 The Seven Axioms of the Cathedral

The crown theorem reports exactly 5 axioms via `#print axioms` (3 kernel + 2 mathematical):

#	Axiom	Origin	Tier
1	`propext`	Lean kernel	—
2	`Classical.choice`	Lean kernel	—
3	`Quot.sound`	Lean kernel	—
4	`critical_line_mellin_variance`	Cathedral	Mellin
5	`rh_zeta_lower_bound`	Cathedral	Hadamard

Both mathematical axioms are provable in ZFC by standard methods. They are formalization gaps, not foundational gaps.

2.3 Constructivity Analysis

The converse ($d_N^2 \to 0 \implies$ RH) is partially constructive: the bound $d_N^2 \geq \delta > 0$ is computed explicitly given ρ; only the existential over ρ in ¬RH is non-constructive. The forward direction is inherently classical, requiring the Montgomery–Vaughan MVT and Perron's formula.

2.4 The de Bruijn Criterion

Component	Size (LOC)
Lean kernel (type-checker)	∼5,000
Lean elaborator + tactics	∼100,000
Mathlib library	∼1,400,000
Cathedral	∼50,623

Only the kernel (5,000 LOC) is in the trusted base.

3 The Axiom-Theorem Gradient

The Cathedral introduces a foundational concept: the *axiom-theorem gradient*.

1. **Kernel axioms**: Trusted by design. (3)

2. **Crown axioms**: On the critical path. (2)

3. **Off-path axioms**: Don't affect the crown. (102)

4. **Theorems**: Machine-verified from Mathlib.

5. **Archived axioms**: Previously active, now proved. (11)

This gradient makes progress visible. Each axiom elimination moves a statement from category 2/3 to category 4. The reduction from 56 crown axioms to 2 is a deleveraging of the proof's balance sheet.

4 Mathlib APIs That Worked Well

4.1 Interval Integrals

Mathlib's `MeasureTheory.integral` and `intervalIntegral` work well for L^2 norms on $(0, 1)$. Key lemma: `integral_nonneg` with `sq_nonneg` for $d_N^2 \geq 0$.

4.2 Matrix API

Mathlib's matrix library is excellent for Gram matrices. `Matrix.PosDef.det_pos` connects positive definiteness to invertibility. `Matrix.nonsing_inv` provides the inverse.

4.3 Spectral Theory

For eigenvalue bounds, use the Rayleigh quotient approach via `Matrix.IsHermitian` (which handles the real case via `starRingEnd`).

4.4 Convergence

When possible, avoid `Filter.Tendsto` and `Filter.atTop`. Algebraic bounds with explicit inequalities are easier to formalize.

5 Mathlib APIs That Required Workarounds

5.1 Infinite Series

`tsum` requires `Summable` hypotheses difficult to establish for Dirichlet series. We used truncated `Finset.sum` with explicit error bounds.

5.2 Fractional Part

`Int.fract` exists but its API is sparse for analytic number theory (integrability, measurability). Expect to prove many basic properties yourself.

5.3 Measure Theory Coercions

Moving between Bochner and Henstock–Kurzweil integrals requires care via `intervalIntegrable_iff_integra`

6 Five Mathlib Gaps Requiring Axioms

Gap	What's Missing	Difficulty
Analytic continuation	Mellin integral extension	Medium
Contour integrals	Vertical line integration	Hard
Mertens function	$M(x) = \sum_{n \leq x} \mu(n)$	Medium
Montgomery–Vaughan	Dirichlet polynomial MVT	Hard
Dedekind sums	Off-diagonal Vasyunin	Medium

These five additions would eliminate 80% of the Cathedral's axioms.

7 Twelve Essential Mathlib Lemmas

#	Lemma	Use
1	`integral_nonneg`	$\int f \geq 0$ when $f \geq 0$
2	`integral_mono_of_nonneg`	Bounding integrals
3	`sq_nonneg`	$x^2 \geq 0$
4	`Matrix.PosDef.det_pos`	Gram invertibility
5	`Matrix.nonsing_inv`	Matrix inversion
6	`Real.log_le_sub_one_of_le`	Log inequalities
7	`Real.rpow_natCast`	Power coercion
8	`Finset.sum_le_sum`	Bounding finite sums
9	`riemannZeta_ne_zero_of_one_le_re`	$\zeta(s) \neq 0$
10	`integral_univ_inv_one_add_sq`	$\int (1+t^2)^{-1} = \pi$
11	`MonotoneBounded.tendsto`	Monotone convergence
12	`Nat.Coprime.gcd_eq_one`	GCD properties

8 Project Structure

8.1 The Layered Module Pattern

1. **Layer 0**: Definitions. No theorems, no axioms.
2. **Layer 1**: Independent mathematical foundations.
3. **Layer 2**: Domain-specific modules (no cross-deps).
4. **Layer 3**: Bridges connecting Layer 2 modules.
5. **Layer 4**: Assembly (crown theorem).

8.2 The Axiom Registry Pattern

Maintain `Axioms.lean` documenting every axiom. Run `#print axioms` on the crown theorem after every structural change. Discrepancies are bugs.

8.3 Sorry Hygiene

1. Never `sorry` on the critical path.
2. Document every `sorry` with a comment.
3. Track sorry count as a project metric.

The Cathedral maintains: 0 sorry on the crown path.

9 Tactic Patterns

- **linarith-norm_num**: For numerical inequalities.
- **ring-then-bound**: Algebra then analysis.
- **convert-congr**: Escape hatch for type unification. Use as last resort.

10 Deprecated API Traps

Deprecated	Replacement	Status
`isHermitian_iff_isSymmetric`	(none yet)	Kept
`Integrable.bdd_mul'`	`Integrable.bdd_mul`	Updated
`measurable_of_continuousOn`	`Continuous.measurable`	Updated

When Lean warns "deprecated," check whether the replacement exists in your Mathlib version.

11 Verification Checklist

☐ `lake build` passes with zero errors.

☐ `#print axioms crown_theorem` shows only expected axioms.

☐ No `sorry` on the crown path.

☐ All sorry markers documented.

☐ Ghost axiom detection passes.

☐ Numerical validation agrees with formal claims.

☐ All deprecated API calls documented.

12 Conclusion

Formalizing analytic number theory in Lean 4 is feasible today, but requires significant infrastructure work. The main gaps are in complex analysis (contour integrals, analytic continuation) and specialized number theory (Mertens function, Dirichlet polynomial bounds). The Cathedral took 32 days and produced 50,623 lines.

The foundational analysis shows the proof inhabits ordinary mathematics (ZFC), requires no large cardinals, and achieves foundational transparency: every assumption is named, classified, and tracked. The trust chain is auditable.

> *A proof is not just a certificate of truth. It is a map of trust, showing exactly where certainty ends and faith begins.*

References

[1] L. de Moura et al., *The Lean 4 Theorem Prover*, CADE-28, 2021.

[2] The Mathlib Community, *Mathlib4*, https://github.com/leanprover-community/mathlib4.

[3] B. Werner, *Sets in Types, Types in Sets*, Proc. TACS 1997.

[4] M. Carneiro, *The Type Theory of Lean*, MSc thesis, CMU, 2019.

[5] N.G. de Bruijn, *The Mathematical Language AUTOMATH*, Springer LNCS 125 (1970).

[6] J.R. Gochanour, *The Cathedral: A Machine-Verified Reduction of the Riemann Hypothesis*, 2026.